



# tecnun

CAMPUS TECNOLÓGICO DE LA UNIVERSIDAD DE NAVARRA  
NAFARROAKO UNIBERTSITATEKO CAMPUS TEKNOLOGIKOA  
Escuela Superior de Ingenieros · Ingeniarien Goi Mailako Eskola

## ● C++ Laboratory Lessons

*Lesson N° 3*



## *Computer Programming II*

*Dr. Paul Bustamante*



## OUTLINE

|                                                              |   |
|--------------------------------------------------------------|---|
| OUTLINE.....                                                 | 1 |
| 1. Introduction.....                                         | 1 |
| 1.1 Exercise 1: Distance calculation .....                   | 1 |
| 1.2 Exercise 2: Calculating the sum of two matrixes .....    | 1 |
| 1.3 Exercise 3: Working with Fractions using structures..... | 2 |
| 1.4 Exercise 4: Distance calculation with structures.....    | 2 |
| 1.5 Exercise 5: Standard deviation calculation .....         | 3 |
| 1.6 Exercise 6: Series with C++ .....                        | 3 |

### 1. Introduction.

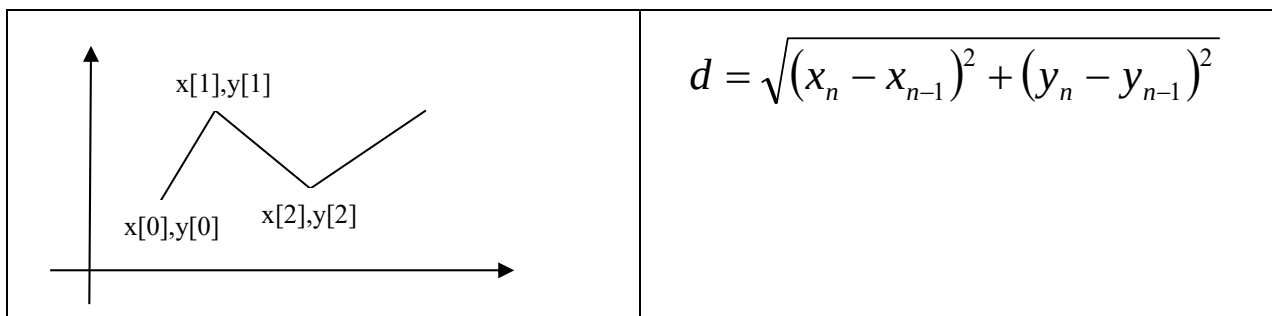
As we have been doing so far, the first exercise of this lesson should be done alone, so you can gain more experience in programming in C++ . In this lab session, you must use the arrays and strings.

#### 1.1 Exercise 1: Distance calculation

This exercise consists of making a program that calculates the distance between a **number of points** (x, y) given by the user. These values must be stored in two arrays (x,y):

```
const int NUM=10;      //max value
double x[NUM], y[NUM];
```

In addition, you must calculate which is the shortest distance found and which the points that distance corresponds to are. Remember that the formula used to calculate the distance between two points is:



#### 1.2 Exercise 2: Calculating the sum of two matrixes

This exercise consists of making a program that allows you to calculate the sum of two square matrixes. The maximum number of rows and columns is defined in the program.

```
matC = matA + matB;
```

You must create a project called **Exer2** and the file **matrix.cpp**, which will contain the following code:

```
//Matrixes calculation - Tecnun 2021
#include <iostream>
using namespace std;
#include <math.h>
#define NUM 10
```

```

void main()
{
    double matA[NUM][NUM], matB[NUM][NUM], matC[NUM][NUM];
    int num=0;

    cout << "Enter the number of rows (=cols):?";
    cin >> num;
    cout << "Enter the values of matA and matB:";
    //Implement your code here: matC = matA + matB;
    //You must use the "for" loop.
    cout << "The values of matC are:"
    //Print the results of matC on the screen
}

```

### 1.3 Exercise 3: Working with Fractions using structures

In this exercise you will learn how to use structures in C++.

The program must ask for the data of two fractions in order to multiply them. At the end, the program must print the results on the screen. You must implement the second part in order to **add these two fractions**. See the program in the next lines:

```

// Tecnun - PBM @ 2021
#include <iostream>
using namespace std;

struct Fraction{
    int num;           //numerator
    int den;           //denominator
};

void main()
{
    Fraction f1,f2,f3;

    cout << "Give Num & Den for f1:?";
    cin >> f1.num >> f1.den;
    cout << "Give Num & Den for f2:?";
    cin >> f2.num >> f2.den;
    //f3 = f1 * f2
    f3.num = f1.num * f2.num;
    f3.den = f1.den * f2.den;
    //print f3
    cout << "F3 = " << f3.num << "/" << f3.den << endl;
    // Now you must implement the code to ADD these two fractions
    // And print the result
    // f3 = f1 + f2;
}

```

### 1.4 Exercise 4: Distance calculation with structures

In this exercise, you must try to do the **Exercise 1** using structures. First, you must create the following structure:

```

struct Point{
    int x;           //X
    int y;           //Y
};

```

Then, you must create an array of structures, like this:

```

const int NUM=10;    //max value
Point Pts[NUM];

```

### 1.5 Exercise 5: Standard deviation calculation

This exercise allows you to introduce a bit in the statistical calculation, and for that, you will implement the standard deviation calculation. As you know, the standard deviation is a value that indicates the amount of dispersion among various samples (**zero** when all are equal). The standard deviation's formula is the following one:

$$\sigma = \sqrt{\frac{\sum_{i=0}^{n-1} (x_i - \bar{x})^2}{n-1}}, \text{ where } \bar{x} = \frac{\sum_{i=0}^{n-1} x_i}{n}$$

Following, the overall structure of the program will be given and you will have to add the necessary code:

```
//Average and standard deviation calculation @Tecnun2021
#include <iostream>
using namespace std;
#include <math.h>

void main()
{
    int Num=0;
    double data[100]; //max number is 100
    double ave, dev;

    // Add here your own code. First ask 'Num' (check<100), next
    // ask for the data to be stored in the vector 'data'
    // Calculate the average and the Deviation
    ave = . . .; //write your code here
    dev = . . .; //write your code here

    cout << "\tAverage value:" << ave << endl;
    cout << "\tStandard Deviation:" << dev << endl;
}
```

### 1.6 Exercise 6: Series with C++

This exercise consists of calculating the value of the trigonometric function Sine, whose expansion in Taylor series is:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \dots \quad (x \text{ in radians})$$

The exercise must be done in **two** ways: asking for the **Number** of Iterations and asking for the amount of **Error**. You must ask to the user which way he/she wants to execute the program. Next, these two modes of execution are detailed:

1. You must write a code that receives two values, the **angle** and the **number of iterations**, and has to print on the screen the result of the series (Sine value).
2. You must write another piece of code that receives two values: the **angle** and the amount of **error** between the iteration and the next one. The program must print on the screen the value of the **sine** function and the **Number** of Iterations.

You must create a new project called **Ejer6** and the file **FuncSin.cpp** for the code.